305432 เครือข่ายเซนเซอร์ (Sensor Networks) · Assignment Paper 3

Raft Protocol for Fault Tolerance and Self-Recovery in Federated Learning

การใช้โปรโตคอล Raft เพื่อความทนทานต่อความผิดพลาดและการกู้คืนตนเองในการเรียนรู้แบบสหพันธ์

Rustem Dautov, Erik Johannes Husom · SINTEF Digital, Oslo, Norway

2024 IEEE/ACM 19th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), pp. 110-121

IEEE Doc ID: 10556381 **DOI: 10.1145/3643915.3644093**

นำเสนอโดย **นายธีรภัทร์ ภู่ระยา** ลำดับที่ 10 · รหัสนิสิต 66362416

ที่มาและปัญหาความท้าทาย (Research Challenges & Problem Statement)

Federated Learning ทำงานอย่างไร (Section 2)

1. **Model initialisation:** aggregator สร้างโมเดลตั้งต้น
2. **Local training:** worker ฝึกด้วยข้อมูลของตนเอง ข้อมูลดิบไม่ออกจากเครื่อง
3. **Model update:** worker ส่งเฉพาะพารามิเตอร์ที่เปลี่ยน
4. **Aggregation:** aggregator รวมพารามิเตอร์ เช่น หาค่าเฉลี่ย
5. **Iteration:** ทำขั้นที่ 2 ถึง 4 ซ้ำหลายรอบ

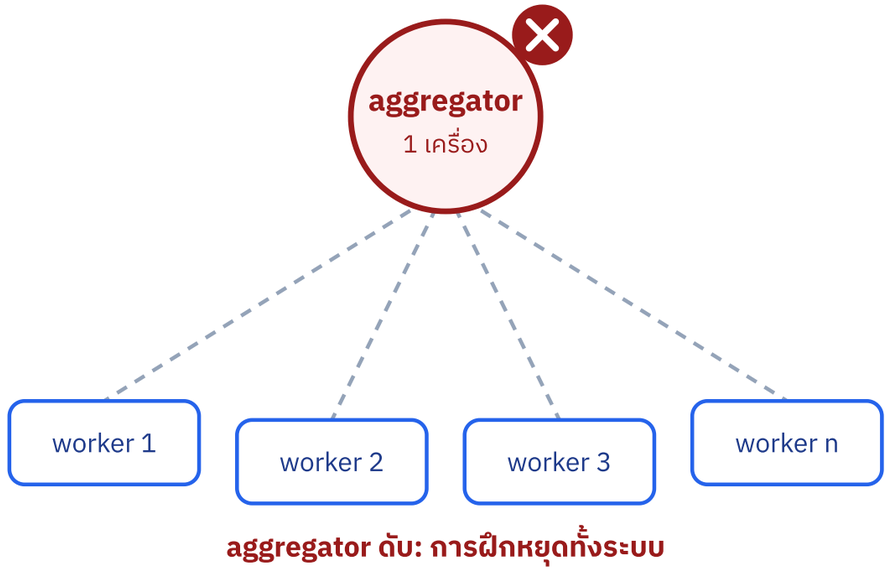
Single Point of Failure (Section 2.1)

จุดอ่อน	การรวมโมเดลยังอยู่ที่ **aggregator เครื่องเดียว**
สาเหตุที่ดับ	ฮาร์ดแวร์เสีย เครือข่ายขาด หรือเหตุไม่คาดคิด
ผลกระทบ	การฝึกหยุดทั้งระบบ และความคืบหน้าที่ยังไม่บันทึกหายไป

ทางเลือกเดิมและเหตุผลที่ผู้เขียนไม่เลือก (Section 3)

Gossip / swarm learning	ไม่มีจุดกลาง แต่ยังซับซ้อนเกินกว่าจะใช้ซ้ำได้ทั่วไป
DLT (blockchain)	ใช้พลังประมวลผล พื้นที่ และพลังงานเกินกำลังอุปกรณ์ IoT
Behera et al.	ใช้ Raft เลือกผู้นำใหม่ แต่ไม่สำเนาสถานะการฝึก

จุดอ่อนของโครงสร้างแบบ star



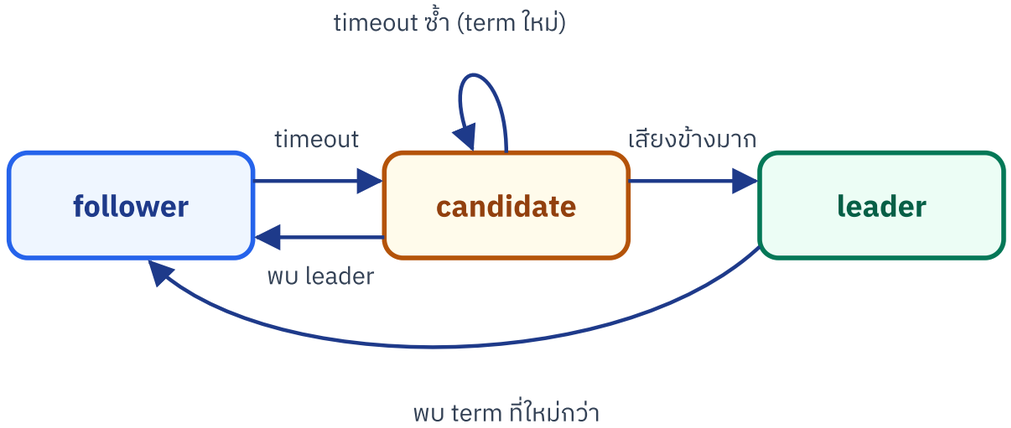
ภาพที่ 1: โครงสร้างแบบ star ของ Federated Learning (สรุปจาก Section 2.1)

ข้อกำหนดของทางแก้ (Section 1)

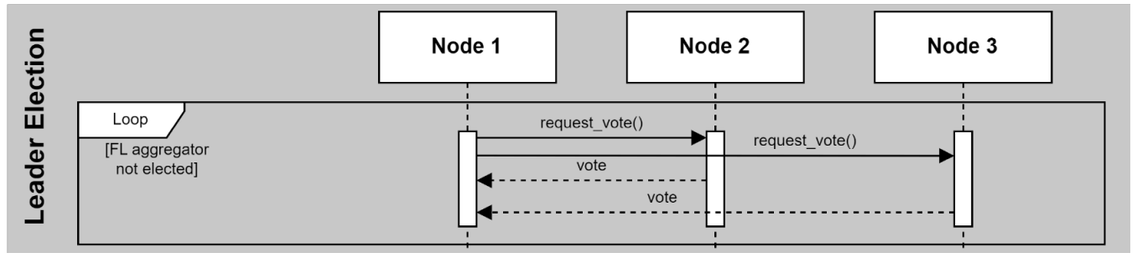
- สำเนาสถานะส่วนกลางของคลัสเตอร์ FL ไปยังทุกโหนดอย่างต่อเนื่อง
- โหนดใดก็ได้ต้องขึ้นเป็น aggregator แทนได้เมื่อเครื่องเดิมดับ
- ผนวกกับเฟรมเวิร์ก FL เดิมได้โดยไม่เพิ่ม software stack

สถาปัตยกรรม Raft-FL และกลไก Leader Election

สถานะของโหนดใน Raft



ภาพที่ 2: การเปลี่ยนสถานะ follower, candidate และ leader ตามนิยามของ Raft [Ongaro and Ousterhout, 2014]



ภาพที่ 3: ช่วง Leader Election ใน Fig. 2 ของบทความ (หน้า 117)

1 Leader Election

follower ที่ไม่ได้รับ heartbeat จนหมด **randomised election timeout** จะเปลี่ยนเป็น candidate เพิ่มหมายเลข **term** แล้วส่ง request_vote ผู้ที่ได้**เสียงข้างมาก**ของสมาชิกทั้งหมดเป็น leader

2 Heartbeat

leader ส่งสัญญาณเป็นระยะ เมื่อสัญญาณเงียบนานเกิน timeout จะเริ่มการเลือกตั้งรอบใหม่

3 Log Replication

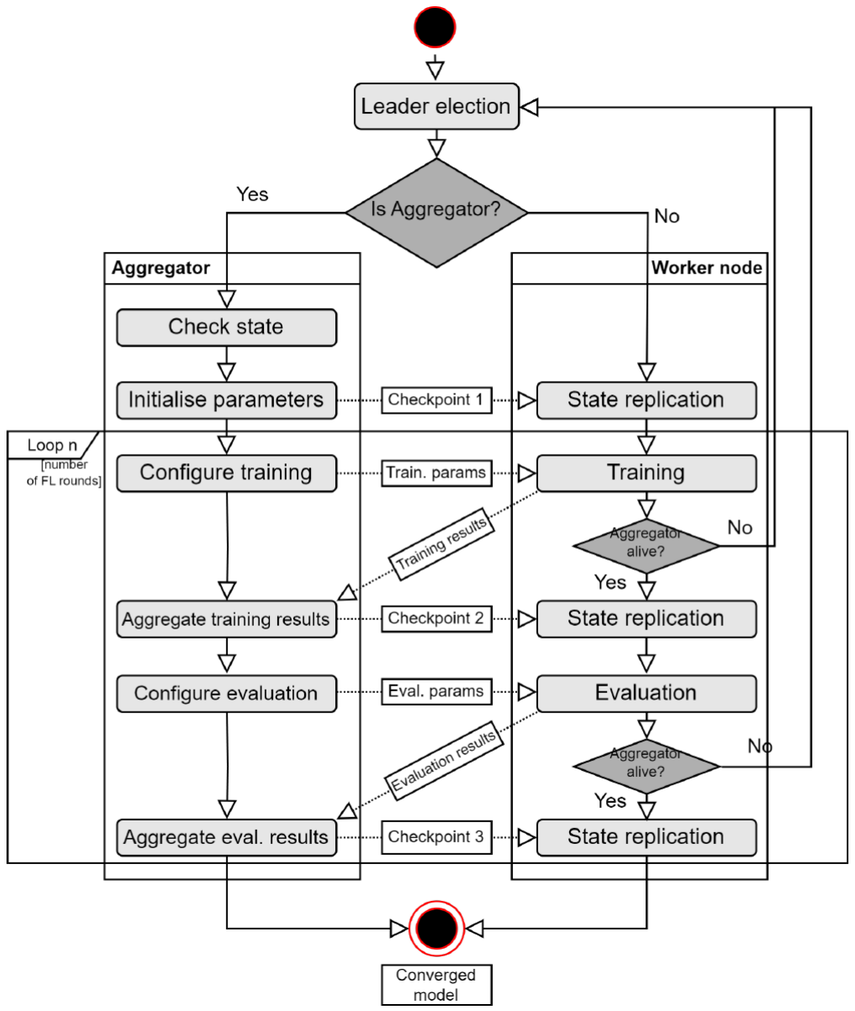
leader ส่ง log entry ให้ follower และ **commit เมื่อเสียงข้างมากตอบรับ** ในบทความนี้ log คือ checkpoint ของพารามิเตอร์

```
// quorum of an N-node Raft cluster
Q = floor(N/2) + 1    f_max = floor((N-1)/2)
// testbed N = 11 -> Q = 6, f_max = 5
```

การจับคู่กับ Federated Learning

Raft leader = FL aggregator และ follower = worker ทุกโหนดรันโค้ดชุดเดียวกัน (Section 5.3)

กลไก Checkpoint Logging และ Self-Recovery Workflow



ภาพที่ 4: Algorithm of a Raft-enhanced FL node (Fig. 1 ของบทความ หน้า 116)

ระดับการกู้คืน 5 ระดับ (Table 1, หน้า 114)

ระดับ	สิ่งที่สำเนา	ผลเมื่อ aggregator ดับ
Level 0	ไม่มี	ความคืบหน้าหาย
Level 1	พารามิเตอร์บนดิสก์ของ aggregator เดิม	ต้องรอเครื่องเดิมกลับมา
Level 2	ไม่มี แต่เลือกผู้นำใหม่ด้วย Raft	ฝึกใหม่ตั้งแต่ต้น
Level 3	**พารามิเตอร์ที่รวมแล้ว (บทความเลือก)**	**ผู้นำใหม่ฝึกต่อจาก checkpoint**
Level 4	เพิ่มพารามิเตอร์ของ worker ทุกตัว	ครบที่สุด แต่ซ้ำซ้อนมาก

Self-Recovery Workflow (Section 5.3)

1 Check state

aggregator ตรวจสถานะที่สำเนาไว้: จำนวนรอบที่ฝึกแล้ว รอบที่เหลือ และพารามิเตอร์ล่าสุด

2 Checkpoint 3 จุด

หลังตั้งค่าพารามิเตอร์ หลังรวมผลการฝึก และหลังรวมผลการประเมิน โดยเขียนลง **replicated key-value** ของ PySyncObj ที่คัดลอกแบบ asynchronous

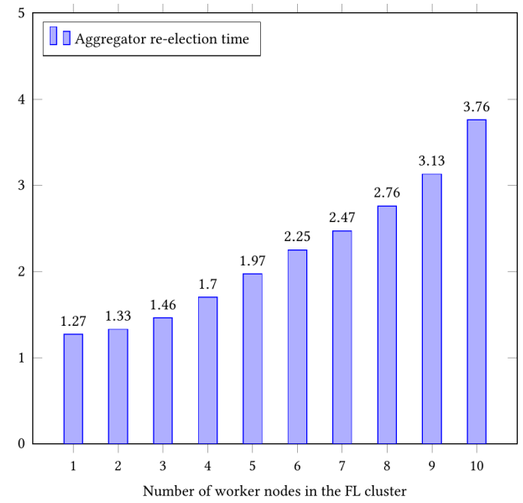
3 Aggregator alive?

worker ตรวจหลังทุกรอบการฝึกและการประเมิน ถ้า aggregator ไม่ตอบ ทุกโหนดกลับไปเลือกตั้งใหม่

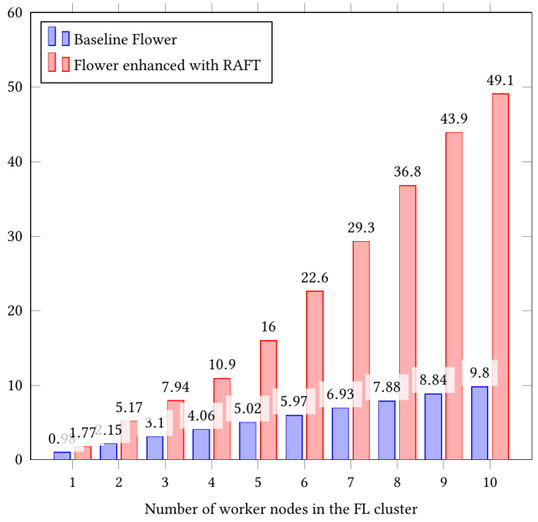
4 Resume

aggregator คนใหม่ใช้พารามิเตอร์จาก checkpoint ล่าสุด ไม่ต้องเริ่มฝึกใหม่ตั้งแต่ต้น

ผลการทดลองเชิงประจักษ์บน Raspberry Pi 11 บอร์ด



ภาพที่ 5: Fig. 3 เวลาเลือก aggregator ใหม่ (s)



ภาพที่ 6: Fig. 4 ทราฟฟิกต่อ 1 รอบการฝึก (MB)

Testbed (Section 6, 6.1)

ฮาร์ดแวร์
Raspberry Pi 3 จำนวน 11 บอร์ด (1 aggregator + worker สูงสุด 10)

งานเรียนรู้
CNN บน CIFAR-10, update ประมาณ 250 kB

การทดลองที่ 1
จงใจปิด aggregator แล้ววัดเวลาเลือกใหม่

เครือข่ายและซอฟต์แวร์
wireless LAN, Raspbian Bullseye 64-bit, Python 3.9

Baseline
Flower PyTorch quick-start tutorial

การทดลองที่ 2
iftop แยก port 8080 (FL) และ 5000 (Raft), 1 รอบการฝึก

1.27-3.76 s
เวลาเลือกใหม่ที่ 1 ถึง 10 worker

49.1 MB
ทราฟฟิกแบบมี Raft ที่ 10 worker (เดิม 9.8 MB)

อัตราส่วนทราฟฟิกแบบมี Raft ต่อ Flower เดิม

worker	Flower เดิม (MB)	มี Raft (MB)	อัตราส่วน*
1	**0.98**	**1.77**	**1.81x**
2	2.15	5.17	2.40x
3	3.10	7.94	2.56x
4	4.06	10.9	2.68x
5	5.02	16.0	3.19x
6	5.97	22.6	3.79x
7	6.93	29.3	4.23x
8	7.88	36.8	4.67x
9	8.84	43.9	4.97x
10	**9.80**	**49.1**	**5.01x**

* คำนวณจากป้ายตัวเลขบน Fig. 4 (หน้า 118)

ข้อสังเกตจากตัวเลขในกราฟ

- "approx. 5 times" ตรงเฉพาะที่ **9 ถึง 10 worker** (1 worker = 1.81 เท่า)
- เวลาเลือกใหม่เพิ่ม **2.96 เท่า** แต่ยังต่ำกว่า 4 วินาที
- ไม่มีค่า SD และไม่วัดความแม่นยำหลังกู้คืน

คุณูปการและการเชื่อมโยงกับเครือข่ายเซนเซอร์และ Distributed Event Streaming

คุณูปการทางวิชาการ (Academic Contributions)

1 กรอบระดับการกู้คืน 5 ระดับ

แยก Level 0 ถึง 4 ตามสิ่งที่สำเนา ทำให้เลือกระดับตามต้นทุนได้ และชี้ว่า **Level 3** คุ้มค่าที่สุดสำหรับ SPOF ของ aggregator

Table 1 และ Section 4.2 หน้า 114

2 ต้นแบบบน Flower ที่เปิดซอร์สโค้ด

ผนวก Raft (PySyncObj) เข้ากับโค้ด FL โดยตรง ไม่ต้องใช้ blockchain หรือบริการภายนอก และ**ทุกโหนดรันโค้ดชุดเดียวกัน**

Section 5.3 หน้า 115, github.com/SINTEF-9012/raft_flower

3 ตัวเลขต้นทุนจากฮาร์ดแวร์จริง

วัดเวลาเลือกใหม่และทราฟฟิกที่ 1 ถึง 10 worker บน Raspberry Pi จริง ทำให้เห็นข้อแลกเปลี่ยน**ระหว่างความทนทานกับแบนด์วิดท์**เป็นตัวเลข

Section 6.2, Fig. 3 และ Fig. 4 หน้า 118

การประยุกต์ใช้ในงานวิศวกรรม (Practical Impact)

A เกตเวย์ของเครือข่ายเซนเซอร์

เกตเวย์ที่รวมข้อมูลก็เป็น SPOF แนวทางที่เหมาะคือให้**เกตเวย์ 3 หรือ 5 เครื่อง**เป็นสมาชิก Raft ส่วนโหนดเซนเซอร์ที่ใช้แบตเตอรี่ทำหน้าที่ FL client

สอดคล้องกับ selective state replication, Section 7.1 หน้า 119

B เทียบกับระบบที่ใช้ Raft ในอุตสาหกรรม

	สมาชิก Raft	ที่เก็บ log
Raft-FL	ทุกโหนด FL	หน่วยความจำ
etcd	คลัสเตอร์ 3 หรือ 5 เครื่อง	ดิสก์ (WAL)
Kafka KRaft	controller quorum	ดิสก์ (metadata log)

Apache Kafka KIP-500; Ongaro and Ousterhout (2014)

C Event-driven backend ด้วย Go

งานที่ต้องมีผู้ทำเพียงรายเดียว เช่น scheduler หรือ ordered consumer ใช้รูปแบบเดียวกันได้: **เลือกผู้นำและบันทึก checkpoint ของ offset**

ข้อจำกัดทางวิศวกรรมและบทสรุป (Engineering Limitations & Conclusion)

ข้อจำกัด 3 ข้อ

1 ไม่อภิปรายเงื่อนไข Quorum

คลัสเตอร์ 11 โหนดทนการดับพร้อมกันได้ไม่เกิน **5 โหนด** จุด 1 worker ใน Fig. 3 เหลือ 1 ใน 2 โหนด ซึ่ง Raft มาตรฐานเลือกผู้นำไม่ได้

Section 6.2.1 ระบุการทดลอง 2 ถึง 10 worker (หน้า 116) ขณะที่ Fig. 3 เริ่มที่ 1 worker

2 ขอบเขตการวัดแคบ

วัดทราฟฟิกเพียง **1 รอบแบบ clean run** ไม่มีค่า SD ไม่วัดความแม่นยำหลังกู้คืน และสถานะเก็บในหน่วยความจำเท่านั้น

Section 6.2.2 และ 6.3 หน้า 118 ถึง 119, เชิงอรรถ 11 หน้า 116

3 ข้อมูลทดลองและรูปแบบความผิดพลาด

CIFAR-10 เป็นภาพ ไม่ใช่ time series จากเซนเซอร์ พารามิเตอร์ 250 kB เล็กกว่าโมเดลจริง และ Raft รับมือเฉพาะ **crash fault**

Section 6 หน้า 116 และ Section 6.3 หน้า 119

บทสรุป

< 4 s

กู้คืนตนเองได้เมื่อมีไม่เกิน 10 worker

1.8-5.0x

ทราฟฟิกที่เพิ่มขึ้นเทียบกับ Flower เดิม

1 สิ่งที่บทความพิสูจน์ได้

Raft แก้ SPOF ของ FL ได้ด้วยกลไกมาตรฐาน และผนวกกับเฟรมเวิร์กเดิมได้โดยไม่เพิ่ม software stack

2 ข้อแลกเปลี่ยนหลัก

ความทนทานต้องแลกด้วยการสำเนาข้อมูล ซึ่งแพงขึ้นเร็วกว่าเส้นตรงเมื่อโหนดเพิ่ม และต้องมีโหนดเหลือมากกว่าครึ่ง

3 แนวทางต่อยอด

สำเนาเฉพาะบางโหนด (selective replication) หรือใช้ gossip ตามที่ผู้เขียนเสนอ และวัดซ้ำด้วยข้อมูลเซนเซอร์พร้อมค่า SD